



COFFE: A Code Efficiency Benchmark for Code Generation

YUN PENG, The Chinese University of Hong Kong, China

JUN WAN, Zhejiang University, China

YICHEN LI, The Chinese University of Hong Kong, China

XIAOXUE REN^{*†}, Zhejiang University, China

Code generation has largely improved development efficiency in the era of large language models (LLMs). With the ability to follow instructions, current LLMs can be prompted to generate code solutions given detailed descriptions in natural language. Many research efforts are being devoted to improving the correctness of LLM-generated code, and many benchmarks are proposed to evaluate the correctness comprehensively. Despite the focus on correctness, the time efficiency of LLM-generated code solutions is under-explored. Current correctness benchmarks are not suitable for time efficiency evaluation since their test cases cannot well distinguish the time efficiency of different code solutions. Besides, the current execution time measurement is not stable and comprehensive, threatening the validity of the time efficiency evaluation.

To address the challenges in the time efficiency evaluation of code generation, we propose COFFE, a code generation benchmark for evaluating the time efficiency of LLM-generated code solutions. COFFE contains 398 and 358 problems for function-level and file-level code generation, respectively. To improve the distinguishability, we design a novel stressful test case generation approach with contracts and two new formats of test cases to improve the accuracy of generation. For the time evaluation metric, we propose efficient@k based on CPU instruction count to ensure a stable and solid comparison between different solutions. We evaluate 14 popular LLMs on COFFE and identify four findings. Based on the findings, we draw some implications for LLM researchers and software practitioners to facilitate future research and usage of LLMs in code generation.

CCS Concepts: • **Software and its engineering** → **Automatic programming**.

Additional Key Words and Phrases: Code Generation, Benchmark, Code Efficiency, Time

ACM Reference Format:

Yun Peng, Jun Wan, Yichen Li, and Xiaoxue Ren. 2025. COFFE: A Code Efficiency Benchmark for Code Generation. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE012 (July 2025), 24 pages. <https://doi.org/10.1145/3715727>

1 Introduction

Nowadays, large language models (LLMs) such as GPT-4 [62] and Llama3.1 [55] have demonstrated great ability to solve different software engineering tasks. With the ability to follow instructions [12, 56, 65, 83], LLMs can act like human developers, promptly handle the instructions and generate completed code, reviews, or comments. Code generation, which is tasked with converting natural language instructions into executable code, has the potential to significantly enhance the efficiency of software development. It is thus a critical software engineering problem being studied by

^{*}Corresponding author.

[†]Also with Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security.

Authors' Contact Information: Yun Peng, The Chinese University of Hong Kong, Hong Kong, China, ypeng@cse.cuhk.edu.hk; Jun Wan, Zhejiang University, Hangzhou, China, 22451014@zju.edu.cn; Yichen Li, The Chinese University of Hong Kong, Hong Kong, China, ycli21@cse.cuhk.edu.hk; Xiaoxue Ren, The State Key Laboratory of Blockchain and Data Security, Zhejiang University, Hangzhou, China, xxren@zju.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 2994-970X/2025/7-ARTFSE012

<https://doi.org/10.1145/3715727>

many researchers. Researchers have proposed different approaches to make use of LLMs on code generation via prompting engineering [10, 58, 70, 74, 94], multiple-agent cooperation [29, 30, 32, 80, 87, 92], and retrieval augmentation [50, 67, 77, 90, 95].

To facilitate the evaluation of code generation, many benchmarks such as HumanEval [8], MBPP [5], CodeContests [43], and APPS [28] are proposed to evaluate the correctness of generated code solutions, and we refer them as correctness benchmarks. These benchmarks include coding tasks drafted by experienced developers [5, 8] or collected from coding competitions [28, 43], with several test cases for each problem to examine the correctness of LLM-generated code solutions. With the correctness benchmarks, researchers can thoroughly study and further improve the ability of LLMs to generate correct code. Built upon current advanced techniques, powerful LLMs such as GPT-4 have obtained remarkable performance with the Pass@1 of 86.6% on the function-level code generation benchmark HumanEval [8], reported by the EvalPlus leaderboard [47].

However, correctness benchmarks alone are insufficient to comprehensively evaluate LLMs' ability of code generation, especially when these models are increasingly used to generate code solutions for software products [88]. In real-world software development, both correctness and time efficiency are crucial for ensuring software quality. Correct but time-inefficient code can lead to a lot of CWE issues [13]. Recent work [44, 48, 69, 75] on LLM-based code generation steps further to generate correct and efficient code. They directly adopt existing correctness benchmarks and measure the execution time of LLM-generated code solutions to determine the time efficiency. We argue that current correctness benchmarks are not suitable for time efficiency evaluation for the following challenges:

- **Challenge 1: Existing correctness test cases cannot well distinguish the time efficiency of different code solutions.** Test cases in correctness benchmarks usually have small inputs since they aim to cover most corner cases to detect potential logical errors in code solutions. However, such test cases can hardly distinguish the time efficiency of different code solutions since code with different time complexities may cost similar time under small inputs. Therefore, it is necessary to include test cases with larger inputs so that we can better distinguish code solutions with different time efficiency. We refer to such test cases as stressful test cases. Stressful test case generation is not straightforward and cannot be easily handled by current correctness test case generation methods. Stressful test cases usually consume much more execution time, so traditional execution-based test case generation methods with many iterations of complete executions are too time-consuming to be adopted. LLM-based test case generation methods without execution can generate stressful test cases quickly, but they are limited by context windows and can hardly maintain the long inputs in results, threatening the accuracy of stressful test case generation.

- **Challenge 2: Execution time metric is unstable and not comprehensive for time efficiency evaluation.** Unlike correctness evaluation, which can be easily repeated on any computer machine, execution time measurements highly rely on the machine where the experiments are conducted. Shypula *et al.* [75] find that two single time measurements of the code solution on the same environment can differ as much as 1.91×. Unstable execution time measurements threaten the validity of time efficiency evaluation. Besides, previous work [33, 75] regards time efficiency evaluation as independent of correctness evaluation for code generation, but time efficiency evaluation is conducted upon correctly generated code solutions. Using separate metrics to evaluate the correctness and time efficiency makes it hard to distinguish the quality of code solutions with high correctness but low time efficiency and those with high time efficiency but low correctness. Currently, there is no single metric evaluating both the correctness and time efficiency of LLM-generated code solutions.

To address the two challenges above in evaluating the time efficiency of LLM-generated code solutions, we **1) propose a new time efficiency benchmark named COFFE, along with a novel approach STGEN to generate stressful test cases automatically.** Specifically, COFFE is built upon existing correctness benchmarks HumanEval [8], MBPP [5] for function-level code generation and CodeContests [43], and APPS [28] for file-level code generation by adding stressful test cases generated by STGEN. Hence, it contains two splits for function-level and file-level code generation. STGEN implements three phases to improve the accuracy of stressful test case generation. In the first phase, STGEN generates contracts that record the dependencies between inputs, and contracts are then used to guide the test case generation in the second phase. An LLM judge checks conflicts between generated contracts and test cases and rejects incorrect test cases in the third phase. By validating test cases on contracts, STGEN can identify incorrect test cases early and provide feedback for LLMs to help fix them. STGEN also uses expressions and generator functions to replace the raw inputs in the stressful test cases to avoid overlong test cases that hinder the generation of LLMs. Furthermore, we **2) propose a new metric named *efficient@k* that considers both correctness and time efficiency based on CPU instruction count measurements.** *Efficient@k* follows the same logic as *pass@k* [8], and the difference is that it requires a code solution to be correct and faster than the best ground truth solution to contribute. When comparing code solutions and ground truth solutions, we replace execution time with a more stable measurement *CPU instruction count* to conduct a solid comparison.

Experiments demonstrate that STGEN is quite effective in stressful test case generation by correctly generating approximately 99% of test cases with a 96% line coverage. Furthermore, To evaluate the effectiveness of stressful test cases generated by STGEN, the stressful test cases generated by STGEN can much better distinguish the time efficiency of code solutions by achieving the relative standard deviation (RSD) of 27.26% and 17.60% over different function-level and file-level code solutions generated by Llama3.1 [55], largely improving the RSD of 19.05% and 15.73% on the original correctness test cases. This indicates the high quality of COFFE. To verify the stability of CPU instruction count, we compare it with execution time and find that CPU instruction count has a RSD of 0.003%~0.005%, which is 1,000× smaller than that of execution time measurement (2.37%~5.65%). This provides a solid basis for the calculation of *efficient@k*.

Based on COFFE, we evaluate the time efficiency of code solutions generated by ten open-source LLMs and four closed-source LLMs and identify the following important findings:

- The performance of current LLMs drops significantly in efficient code generation, indicating that the code solutions generated by current LLMs are correct but not time-efficient.
- Compared with function-level code generation, code solutions generated by current LLMs are less efficient in file-level code generation.
- Larger LLMs generally perform better in correct code generation but do not significantly outperform smaller LLMs in efficient code generation, indicating larger parameter sizes of current LLMs do not contribute much to efficient code generation.

We summarize the contributions of this paper as follows:

- We build COFFE, a benchmark for evaluating the time efficiency of both function-level and file-level code solutions generated by LLMs.
- We propose STGEN, the first LLM-based stressful test case generation approach that employs contract validation and test cases with expression and generator functions inputs to improve accuracy.
- We introduce a novel metric *efficient@k*, based on stable CPU instruction count measurement, to evaluate the correctness and time efficiency of the LLM-generated code solutions.

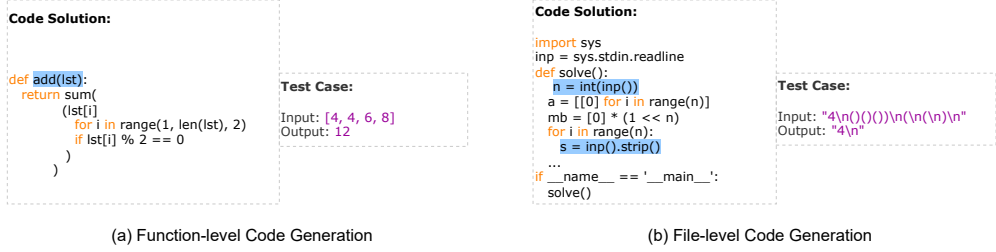


Fig. 1. Examples for function-level and file-level code generation.

- We conduct extensive experiments to evaluate the quality of COFFE, the effectiveness of STGEN, and the ability of current LLMs to generate efficient code.

2 Problem Definition

Currently, there are three types of code generation tasks: function-level, file-level, and repo-level code generation. We mainly focus on the first two types of code generation since repo-level code generation involves different modules in the repositories and third-party dependencies, making it hard to obtain solid time efficiency measurements. To better illustrate the differences between function-level and file-level code generation, we present two examples in Figure 1.

Function-level Code Generation. Function-level code generation takes natural language functionality descriptions as input and generates a single function that satisfies the requirements. The generated function accepts inputs through function parameters. The HumanEval [8] and MBPP [5] benchmarks are designed to benchmark function-level code generation.

Figure 1(a) shows an example function. We observe that the function `add()` only has a parameter named `lst`, and we only need to generate test inputs for this parameter to build a test case. This shows that the number of parameters in functions is determined and functions accept inputs only once from parameters before the function execution. Therefore, **to generate test cases for function-level code generation, we can generate test inputs for each parameter and combine them as a test case.**

File-level Code Generation. File-level code generation generates a complete program file instead of a single function to satisfy specified requirements. The inputs of the program file are managed by *standard input* (*stdin*) related APIs, e.g., `input()`. File-level code generation tasks frequently appear in coding competitions, based on which researchers built Code Contests [43] and APPS [28] benchmarks.

Figure 1(b) shows an example program file. We observe that this code solution accepts inputs in two locations (highlighted in blue). The input in the first location is used to control how many times the input in the second location will take. This indicates that **the number of inputs for program files is not only determined by the code solution but also by the inputs.** This poses great challenges in generating test cases for file-level code generation.

3 Methodology

This section describes how we build the benchmark COFFE, including selecting the coding problems, proposing STGEN to generate stressful test cases for function-level and file-level code generation, and designing a novel time efficiency metric `efficient@k`.

Table 1. The statistics of four sanitized benchmarks we selected to build COFFE. “Ori.”, “Val.” and “Sel.” indicate the original problems, validated problems, and finally selected problems in the benchmarks. The other columns in the table represent the data for the finally selected problems.

Benchmark	#Problem			#Solution/Problem	#Test Case/Problem	Level
	Ori.	Val.	Sel.			
HumanEval	164	164	164	1.00	9.57	Function
MBPP	234	234	234	1.00	3.02	Function
Code Contests	111	106	58	80.26	197.53	File
APPS	5,000	3,106	300	64.36	13.94	File

3.1 Data Preparation

To construct COFFE, we collect problems in the test splits of two existing function-level correctness benchmarks (i.e., HumanEval [8] and MBPP [5]), and two existing file-level correctness benchmarks (i.e., APPS [28] and CodeContests [43]). Each benchmark contains multiple coding problems and provides each problem with a description that explains the requirements in natural language, several ground truth solutions that address the problem, and several test cases that evaluate the correctness of generated code solutions. As there are multiple versions for MBPP, we choose the common subset of the sanitized version [24] and the MBPP+ benchmark verified by EvalPlus [46] as our base benchmark to ensure the highest quality.

With the selected benchmarks, we first validate the problems by checking the potential conflicts of provided test cases and ground truth solutions. Secondly, we select problems that most LLMs could correctly answer to reduce the difficulty of problems for the two file-level benchmarks since a problem is not useful in time efficiency evaluation if no LLM can answer it. We show the statistics of four benchmarks in Table 1.

3.1.1 Problem Validation. To ensure the quality of test cases and ground truth solutions in the four benchmarks, we run the ground truth solutions in the provided test cases and remove 1) ground truth solutions that cannot pass the provided test cases to ensure consistency, 2) ground truth solutions with file operations to keep safety, and 3) problems without valid ground truth solutions and test cases. We show the number of validated problems in each benchmark in the third column of Table 1. All problems in HumanEval and MBPP can be successfully validated, so no problem is removed. For the Code Contests benchmark, we identify five problems with file operations, and we remove them to guarantee the safety of testing environments. For the APPS benchmark, we identify 1,894 problems whose ground truth solutions conflict with the provided test cases. The reason for such conflicts is that the APPS benchmark does not require the output of a code solution to exactly match the expected outputs in test cases to be correct, which differs from the other three benchmarks. We remove the 1,894 problems without exact matches in the APPS benchmark to maintain consistent evaluation standards.

3.1.2 Problem Selection. Current LLMs are quite effective in function-level code generation by achieving a pass@1 of more than 80% in the HumanEval benchmark, as discussed in Sec. 1. However, they perform much worse in file-level code generation since the most powerful LLM has a Pass@1 of 28.5% on the Code Contests benchmark and a Pass@1 of less than 10% on the APPS benchmark [85, 86]. This limits the usage of the full set of the Code Contests and APPS benchmarks because a problem that no LLM can correctly answer does not contribute to the time efficiency evaluation. Therefore, for the validated problems in the two benchmarks, we sample one code solution with temperature 0 on 14 LLMs used in our experiments described in Table 3 and remove 48 and 2,223

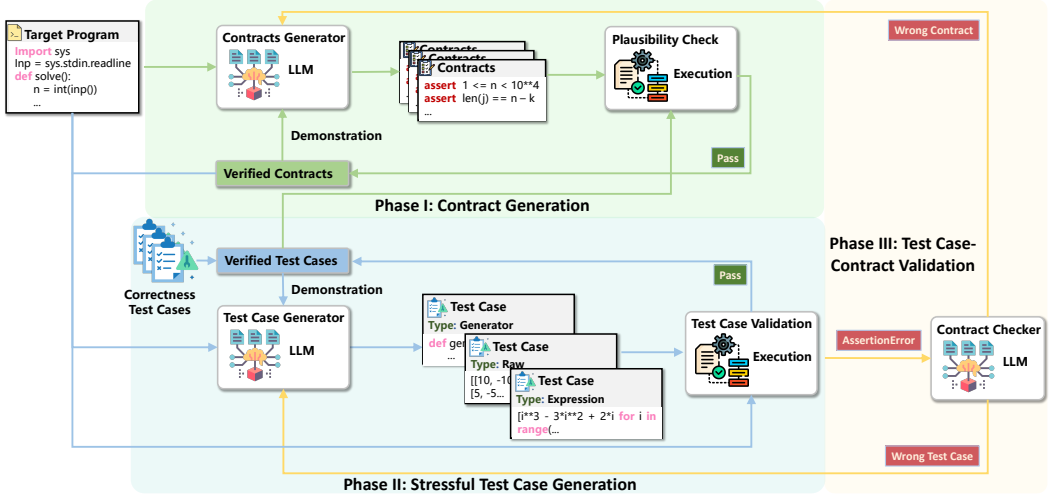


Fig. 2. The overview workflow of STGEN.

problems that code solutions from all LLMs failed in the Code Contests and APPS benchmark, respectively. To balance the number of problems in the function-level split and file-level split of COFFE, we further select 300 problems in the APPS benchmark for which more LLMs can generate correct code solutions. We show the number of selected problems from the four benchmarks and associated statistics in the 4~6 columns of Table 1.

3.2 Stressful Test Case Generation: STGEN

With the selected problems, we propose a novel LLM-based approach STGEN to generate stressful test cases automatically. In contrast to current LLM-based test case generation methods [7, 41, 45, 66, 72, 73], STGEN aims to generate test cases to evaluate the time efficiency of code solutions under extreme conditions rigorously. This inherently requires constructing exceptionally long and intricate inputs that can hardly be handled by LLMs directly, leading to unsatisfactory accuracy, i.e., the proportion of correctly generated stressful test cases is low.

3.2.1 Overview. To improve the accuracy of stressful test case generation, STGEN introduces contracts to guide the test case generation and validate the generated test cases. Contracts are collections of assertion statements that record the type, scale, and internal constraints between the inputs. Providing contracts in the test case generation process can help LLMs understand the dependencies between test inputs. Besides, STGEN can easily identify incorrect test cases from the assertion errors contracts raise. To avoid overlong stressful test cases that hinder the performance of LLMs, we design two new formats of test cases by reformulating the test case generation task into a code generation task: *expression test cases* and *generator test cases*. Different from raw test cases that directly provide test inputs, expression and generator test cases contain code to generate test inputs, which greatly shortens the length of test cases.

We present the overview of STGEN in Figure 2. STGEN does not directly generate stressful test cases. Instead, it decomposes the task into three phases: 1) contract generation, 2) stressful test case generation, and 3) test case-contract pair check. In the first phase, STGEN generates contracts by analyzing the target program, i.e., the ground truth solution for each problem in the benchmark. The generated contracts are then provided as demonstrations for stressful test case generation in the second phase, in which STGEN generates expression and generator test cases instead of raw test cases. Since contracts are also generated and there is no guarantee of their correctness, STGEN

enters the third phase if the number of *AssertionError* occurrences for a certain contract exceeds a threshold. In the third phase, STGEN implements an LLM judge to determine the responsibility for conflicts between generated contracts and test cases. The contracts or test cases that are judged to be incorrect will be sent back for regeneration. This iterative process allows the generation of contracts and stressful test cases to mutually reinforce each other.

3.2.2 Phase I: Contract Generation. In the first phase, STGEN inserts assertion statements that check the preconditions of inputs as contracts into the target program, such as *assert $n > 0$* . The contracts ensure that the inputs meet the required specifications in format (e.g., variable type), scale (e.g., input length, order of magnitude), and intrinsic constraints (e.g., right triangle side lengths).

The benefits of inserting contracts before stressful test case generation are twofold: 1) **Knowledge Enrichment.** Contracts explicitly indicate the functionality of the target program and the dependencies between inputs, which can help LLM better understand natural language descriptions provided in problems [20, 45]; 2) **Early Validation.** Contracts can identify invalid inputs in test cases at the beginning of program execution and stop the execution-based test case validation process early, which largely improves the efficiency of the test case generation process.

In contract generation, STGEN generates one assertion statement in an iteration and combines all assertion statements into a contract. When generating assertion statements, STGEN prompts LLMs to consider the type, scale, and intrinsic constraints between inputs given the target program, existing correctness test cases, and previously generated assertion statements as demonstrations. STGEN implements the same methodology to generate assertion statements for function-level and file-level target programs. However, STGEN employs different strategies to insert assertion statements into target programs, given the differences between the code solutions in function-level and file-level code generation illustrated in Sec. 2.

Function-level Contract Insertion. For function-level target programs with a determined number of inputs, STGEN generates and inserts assertion statements for function parameters at the beginning of the function body. For example, STGEN inserts assertion statements right before the return statements in the function *add()* in Figure 1(a).

File-level Contract Insertion. For file-level target programs with an unknown number of inputs and multiple input locations, STGEN reformulates the contract generation problem into a code editing problem. It first identifies all input locations by checking the related system APIs such as *input()* and then inserts assertion statements for each identified input location sequentially. STGEN inserts assertion statements right after the input locations in most cases. However, as input locations in loops generally assign values for generic types such as *list* and *dict*, STGEN inserts assertion statements after the entire loop where the assignments are complete to check the fully assigned types. For example, STGEN identifies two input locations highlighted in blue in Figure 1(b). STGEN first generates assertion statements for the input that assigns values to variable *n* and inserts them right after the assignment. STGEN then generates assertion statements for the second input in the loop, and this time, it inserts assertion statements after the entire *for* loop.

To improve the correctness of generated assertion statements, STGEN tests all generated assertion statements against the correctness test cases each time it inserts a new assertion statement. If the current assertion statement fails on the test cases, STGEN only removes the current assertion statement and regenerates a new one while maintaining the assertion statements correctly generated in previous iterations. The iteration ends until no new assertion statements are generated or a maximum iteration number is reached.

3.2.3 Phase II: Stressful Test Case Generation. With the generated contracts as demonstrations, in the second phase, STGEN generates stressful test cases. Unlike correctness test case generation, it is quite challenging to generate stressful test cases because LLMs must generate test cases of maximal

length and complexity within the constraints of its finite context window while simultaneously ensuring adherence to intrinsic input constraints specified by contracts. Correctness test cases in current benchmarks [5, 8, 28, 43] are raw test cases that directly provide the values for test inputs. However, due to the limited context window size, it is infeasible to directly generate overlong raw test cases for time efficiency evaluation. For example, it is hard for LLMs to generate a list with more than a million numbers for stressful tests. To address this challenge, we introduce two new formats of stressful test cases:

Expression Test Cases. Expression test cases utilize Python expressions to generate test cases, allowing for more complex input generation while maintaining a compact representation within the LLM’s context window. For instance, a list with a million numbers could be easily generated by an expression “`[random.randint(1, 100000) for _ in range(100000)]`”, which is much shorter than listing a million numbers. Expression test cases offer a balance between complexity and conciseness, enabling the creation of structured inputs. They are suitable for function-level test case generation with a determined number of test inputs. To evaluate code solutions on expression test cases, we just need to execute the expressions to get the real test inputs before the code execution.

Generator Test Cases. Generator test cases are Python functions that output the test inputs. It is quite useful for creating stressful test cases that require intricate logical relationships or patterns that are difficult to express in single expressions. For example, it is suitable for file-level code generation where the number of inputs is undetermined. Expression test cases cannot handle this since we do not know how many expressions should be generated.

To generate expression and generator test cases, STGEN prompts the LLMs with contract, verified generated test cases as demonstrations, so LLMs can learn the dependencies between inputs as well as the specific formats of the expected test cases. The generated test cases are then verified against the previously generated contracts and the target program. Test cases that pass the validation of contracts and the execution of the target program are collected to build COFFE. Verified stressful test cases are also used as demonstrations to help generate the following stressful test cases.

3.2.4 Phase III: Test Case-Contract Pair Check. Although the generated contracts are verified against the existing correctness test cases, correctness test cases do not cover all possible cases and dependencies among inputs, especially in stressful scenarios. Contracts can still make mistakes and induce false positives. During the test case validation, if a generated test case violates the inserted contract, it triggers an *AssertionError*. If the *AssertionError* consistently occurs for multiple test cases, the contract may be incorrect and thereby hinder the entire stressful test case generation procedure. To mitigate this, when the number of conflicts between contracts and test cases (i.e., *AssertionError* occurrences during execution) exceeds a predefined threshold (5 in this paper), the generated test case and the violated contract are paired for further check by an LLM judge checker in the third phase.

The LLM judge takes all accumulated contract-related execution failure pairs as inputs, along with the target program, to analyze and determine the validity of the contracts and the test cases. The judge reviews the violated contract with exact stressful inputs, rethinks the correctness of the generated contract, and determines the root cause of conflicts. Once the root cause is identified, the relevant judgment results and corresponding failure pairs are sent back to the previous phases for regeneration. By providing feedback for incorrect contracts or test cases, STGEN enhances the robustness of the test case validation and enables the improvements between contract generation and test case generation. To prevent duplicate judgments, once the LLM judge determines that a contract is valid in the third phase, it will not be checked again, and test cases that fail the validation of this contract will be directly rejected in the future.

3.3 Time Efficiency Metric: Efficient@k

Previous work [33, 75] intuitively adopts execution time as the performance measurement to evaluate the time efficiency of LLM-generated code. However, execution time measurements could be affected by many factors, such as process scheduling and disk I/O, so it is not stable enough to make a solid comparison between the time efficiency of different code solutions. In this section, we propose to use *CPU instruction count* to replace execution time to measure the time efficiency of code solutions stably. Based on CPU instruction count measurements, we propose a new metric *efficient@k* to evaluate both the correctness and time efficiency of code solutions.

3.3.1 CPU Instruction Count. To find a more stable measurement to replace execution time, we first look into the factors contributing to the execution time. Patterson and Hennessy [68] define the CPU time cost by a program through the following equation.

$$\text{CPU Time} = [\text{Instruction Count}] \times [\text{Clock per Instruction}] \times [\text{Clock Cycle Time}] \quad (1)$$

From the equation, the CPU time of a program is determined by three factors. While *Clock per Instruction* and *Clock Cycle Time* depend on the physical machine where the program runs, the only factor related to the program is *Instruction Count*. Therefore, if a program has a higher CPU instruction count on the same machine, it is less efficient, and vice versa. Unlike the execution time measurements that could be affected by many factors, CPU instruction count measurements are more stable as CPU instruction count for a program does not increase even if the program execution is slowed or stalled by external factors. It is also straightforward to measure CPU instruction count using the system APIs. For example, Linux provides a command tool named *perf* [21] to support CPU instruction count measurements.

3.3.2 Efficient@k. CPU instruction count is a stable measurement for the time efficiency evaluation of different code solutions. However, its absolute value is not meaningful as the same code solution has different CPU instruction counts in different machines. Besides, it is not comprehensive as it does not measure the correctness of generated code solutions. To address these problems, we propose a new metric named *Efficient@k*, inspired by the design of *pass@k* [8]. We show the original definition of *pass@k* in Equation 2 and the definition of proposed *efficient@k* in Equation 3.

$$\text{pass@k} := \mathbb{E}_{\text{Problems}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right] \quad (2)$$

$$\text{efficient@k} := \mathbb{E}_{\text{Problems}} \left[1 - \frac{\binom{n-c_f}{k}}{\binom{n}{k}} \right] \quad (3)$$

Pass@k is an expectation over all problems in the benchmark for the probability that at least one solution in k samples can pass all test cases. In equation 2, total n solutions are sampled from LLMs instead of only k samples to reduce the variance. By running the sampled code solutions on correctness test cases, we can get the solutions c that can pass all the test cases to estimate the probability of correctness. *Pass@k* is a solid metric with low variance and can be easily reproduced under different platforms.

We follow the idea of *pass@k* when designing *efficient@k*. *Pass@k* requires the correct code solutions c to contribute, while in *efficient@k*, we collect the number c_f of the correct solutions faster than the best ground truth solution to replace c in *pass@k*. Therefore, *efficient@k* evaluates the probability of LLMs to generate correct and fast enough code solutions. *Efficient@k* compares the CPU instruction count of code solutions and ground truth solutions to determine which runs faster. By doing so, *efficient@k* does not consider the absolute values of CPU instruction counts to

Table 2. The statistics of COFFE.

Category	#Problem	#Solution/Problem	#Test Case/Problem	
			Correctness	Stressful
Function-level	398	1.00	5.72	4.99
File-level	358	66.93	43.68	4.95

avoid the impacts of specific systems or machines. With a value range from 0 to pass@k, efficient@k combines correctness and time efficiency evaluation to comprehensively evaluate the quality of code solutions.

3.4 Code Efficiency Benchmark: COFFE

With the stressful test case generation approach STGEN, we add stressful test cases for each problem selected in Sec. 3.1. Specifically, we generate 20 stressful test cases for each problem and measure the CPU instruction count each test case costs. We conduct the measurements 12 times and remove the highest and lowest measurements before calculating the average to ensure the most stable results. In the CPU instruction measurements, we limit the execution time of one single measurement to five seconds so that the measurements for one test case will not exceed one minute. We then rank the average CPU instruction count of each test case and include the five test cases with the highest CPU instruction counts in COFFE. We do not include all generated stressful test cases in COFFE to avoid large time costs in time efficiency evaluation, since stressful test cases generally take much longer time than correctness test cases to execute. We reserve all existing correctness in COFFE to validate the correctness of generated code solutions. We show the statistics of COFFE in Table 2.

4 Experiment Setup

4.1 Research Questions

We focus on the following research questions:

- **RQ1:** How well does CPU instruction count measure time efficiency compared with execution time?
- **RQ2:** How effective is STGEN on stressful test case generation and how well are the generated stressful test cases?
- **RQ3:** How efficient is the code generated by current LLMs?

4.2 Metrics

To evaluate the stability of CPU instruction count (RQ1), we introduce the following metrics:

- **Relative Standard Deviation (RSD):** The ratio of the standard deviation to the mean. We use it to measure how stable a performance metric is on the same code solution (the lower, the better) and how well a test case can distinguish different code solutions (the higher, the better). We use “RSD (-)” when it is used to evaluate stability and “RSD (+)” when it is used to evaluate distinguishability.
- **Pearson Correlation Coefficient:** The ratio between the covariance of two variables and the product of their standard deviations. We use it to measure the linear correlation between two metrics.

To evaluate the quality of stressful test cases and the effectiveness of STGEN (RQ2), we introduce the following metrics:

- **Accuracy:** The proportion of test cases generated by a certain method where the target program does not fail.
- **Line Coverage:** The percentage of executed lines in solutions when executing the test cases.

To evaluate the efficiency of code solutions generated by LLMs (RQ3), we use the following metrics:

- **Pass@k:** The probability that at least one of the top k-generated code samples for a problem passes the unit tests, as illustrated in Sec. 3.3.
- **Speedup:** The ratio $\frac{gt}{o}$ of CPU instruction count of best ground truth solution gt to the CPU instruction count of a code solution o .
- **Efficient@k:** The probability that at least one of the top k-generated code samples for a problem is correct and more efficient than the best ground truth solution, as introduced in Sec. 3.3.

4.3 Baselines

Since there is no previous work on LLM-based stressful test case generation, we select three widely used LLM-based correctness test case generation methods and adapt them into stressful test case generation:

- **Instruction Prompting [79].** Wang *et al.* design several instruction prompt templates to ask LLMs to cover certain lines, branches, or paths of the code in test case generation. We modified their instruction prompt and let LLMs focus on stressful test case generation. This method generates raw test cases.
- **Few-shot Prompting [66].** Few-shot prompting adds several demonstrations to guide LLMs to generate similar test cases. This method generates raw test cases.
- **Generator-based Prompting [49].** Instead of directly generating test cases, this method prompts LLMs to generate a function that derives the test cases. We adapt this method to our stressful test case generation and let LLMs generate functions that produce stressful test cases. This method generates generator test cases.

4.4 Models

To investigate the efficiency of code generated by current LLMs, we select 14 popular models for evaluation. We show the model names, sizes, and context lengths in Table 3. For GPT-3.5 [61] and GPT-4o [63], we use the APIs provided by OpenAI [64] under engines “*gpt-3.5-turbo*” and “*gpt-4o*”, respectively. For DeepSeek V2 [16] and DeepSeek V2 Coder [96], we use the APIs provided by DeepSeek [15] under engine “*DeepSeek-V2-0628*” and “*DeepSeek-V2-0724*”, respectively. For Claude 3.5 Sonnet [4], we use the APIs provided by Anthropic [3] under the engine “*claude-3-5-sonnet-20240620*”. For Gemini 1.5 Pro, we use the APIs provided by Google [25] under the engine “*gemini-1.5-pro*” to generate code solutions. Due to limited computing resources, for open-source models larger than 13B, we use the API provided by Deep Infra [34] to generate code solutions.

4.5 Implementation Details

We conduct all experiments on a Linux machine with Ubuntu 20.04.4 LTS. It has an Intel(R) Xeon(R) Platinum 8358P CPU of 2.60G HZ with 128 cores and 2 TB memory. We use the Coverage.py [6] library to measure the line coverage of test cases, and the Cirron [76] library to measure the CPU instruction count a program consumes.

Table 3. The LLMs we evaluate in this paper. Models highlighted in gray are closed-source models.

Model	Size	Context Size	Model	Size	Context Size
Phi3 [1]	3.8B	128k	MagicCoder [84]	DS-6.7B/CL-7B	16,384
CodeLlama [71]	7B/13B/34B	16,384	Llama3 [54]	8B/70B	4,096
StarCoder [42]	15B	16,384	WizardCoder [53]	15B	2,048
Mixtral [35]	8×7B	32,768	DeepSeek V2 [16]	236B	128k
DeepSeek Coder V2 [96]	236B	128k	Llama3.1 [55]	405B	4,096
Claude 3.5 Sonnet [4]	-	200k	Gemini 1.5 Pro [14]	-	200k
GPT-3.5 [35]	-	16,385	GPT-4o [63]	-	128k

5 Experiment Results

5.1 RQ1: CPU Instruction Count vs. Execution Time

To demonstrate that CPU instruction count is more suitable for time efficiency evaluation than execution time, we focus on two aspects: stability, which evaluates how solid the measurement is, and correlation, which evaluates how close two measurements are.

Stability. To compare the stability of CPU instruction count and execution time measurements, we run the ground truth solutions of the validated problems on the correctness test cases from the four correctness benchmarks. Note that we do not run them on our stressful test cases to ensure a fair comparison since CPU instruction count is involved in building COFFE. We run each solution 12 times and remove the largest and smallest measurements. We then calculate the RSD of the remaining 10 measurements and show the results in the second and third columns of Table 4.

As the experiments are repeated on the same ground truth solution and same test cases, a lower relative standard deviation indicates a more stable measurement. From Table 4, we can observe that execution time has an RSD of about 5% on function-level benchmarks HumanEval and MBPP and an RSD of about 2% on file-level benchmarks Code Contests and APPS. On the contrary, CPU instruction count has a more than 1000× smaller RSD (0.003%~0.005%) than execution time on four benchmarks. This indicates that the ten measurements of CPU instruction count almost remain the same on the same program, and CPU instruction count is quite stable in measuring time efficiency.

Correlation. To validate the linear correlation between CPU instruction count and execution time, as described in Equation 1, we calculate the Pearson correlation coefficient between CPU instruction count and execution time, as shown in the last column of Table 4. We find that the correlations of the two measurements on all benchmarks are very close to 1.0. This indicates that the two measurements are linearly correlated and verifies the correctness of Equation 1 as the other two factors *Clock per Instruction* and *Clock Cycle Time* do not change in the same testing environment. Therefore, we can replace execution time with CPU instruction count to measure time efficiency.

Answer to RQ1: CPU instruction count is more suitable to evaluate time efficiency since it is much more stable than execution time by achieving a 1000× smaller RSD of 0.003%~0.005%, and it is linearly correlated with execution time with Pearson correlation coefficient of 0.96~1.0.

5.2 RQ2: Effectiveness of STGEN and Distinguishability of Stressful Test Cases

To answer RQ2, we study the effectiveness of STGEN on stressful test case generation compared with three widely used LLM-based test case generation baselines. For the generated stressful test

Table 4. Comparison between CPU Instruction Count and Execution time on different benchmarks. “RSD (-)” indicates the average relative standard deviation of a certain metric when running multiple times on the same ground truth solution. It evaluates the stability of different measurements. “Correlation” indicates the Pearson correlation coefficient between CPU instruction count and Execution time.

Benchmark	RSD (-)		Correlation
	CPU Instruction Count	Execution Time	
HumanEval	0.005%	5.65%	1.00
MBPP	0.004%	5.31%	1.00
Code Contests	0.003%	2.37%	0.99
APPS	0.003%	2.47%	0.96

Table 5. Comparison between different test cases. “Correctness” indicates the original correctness test cases. “Instruction”, “Few-shot” and “Generator” indicate the stressful test cases generated by three baselines, respectively. “RSD (+)” indicates the relative standard deviation achieved by test cases on different code solutions generated by two powerful LLMs GPT-4o and Llama3.1. It evaluates the distinguishability of different test cases in terms of time efficiency.

Level	Method	Accuracy	Line Cov.	RSD (+)	
				Llama3.1	GPT-4o
Function	Correctness	-	98.46	19.05%	21.35%
	Instruction	83.67	83.87	20.69%	22.21%
	Few-shot	87.07	81.46	22.32%	20.61%
	Generator	86.91	81.84	22.86%	21.32%
	STGEN	98.64	96.01	27.26%	28.20%
File	Correctness	-	95.68	15.73%	12.99%
	Instruction	84.52	85.29	14.21%	11.04%
	Few-shot	65.17	66.53	12.02%	8.95%
	Generator	94.86	94.79	15.54%	13.17%
	STGEN	98.91	95.17	17.60%	14.79%

cases, we evaluate whether they can better distinguish different code solutions generated by LLMs. We show the main results of the comparison between STGEN and baselines in Table 5.

Effectiveness of STGEN. To study how contracts can improve the accuracy of test cases, we compare STGEN with three baselines without contracts and report the accuracy at the third column of Table 5 for function-level and file-level splits of COFFE. We do not report the accuracy of the original test cases because they are manually drafted. From the table, we observe that STGEN achieves an accuracy of 98.64% and 98.91%, outperforming the baselines by up to 17.89% and 51.77% in function-level and file-level splits, respectively. This suggests that almost all stressful test cases generated by STGEN are correct. Without knowledge enrichment and early validation by contracts, on the contrary, baselines fail to generate about 5%~35% of stressful test cases.

Apart from the accuracy, a correct test case is representative if it can cover most lines of the target program. To ensure the quality of generated stressful test cases, we evaluate the line coverage and report the results in the fourth column of Table 5. We find that STGEN consistently achieves

the highest line coverage of 96.01% and 95.17% for function-level and file-level stressful test cases, respectively. This demonstrates that the stressful test cases generated by STGEN can thoroughly evaluate the time efficiency of the major code logic in target programs. We also note that the line coverage achieved by STGEN is slightly lower than that achieved by the original correctness test cases. This is reasonable because the stressful test cases are much fewer than the correctness test cases in COFFE, as can be seen in Table 2.

Distinguishability of Stressful Test Cases. To evaluate how well the stressful test cases generated by STGEN can distinguish the time efficiency of different code solutions, we sample 20 code solutions from two powerful LLMs, Llama3.1 and GPT-4o, for each problem in COFFE. We then run the sampled solutions on different test cases and collect the CPU instruction count usage. We calculate the RSD on the CPU instruction counts of the sampled 20 code solutions, and a higher RSD indicates better distinguishability. We report the RSD on the code solutions of two models at the fifth and sixth columns of Table 5.

Firstly, we observe that stressful test cases generated by STGEN improve the RSD of original correctness test cases by 43.10% and 32.08% on Llama3.1 and GPT-4o, respectively, at the function level, and the improvements are 11.89% and 13.86% on Llama3.1 and GPT-4o, respectively, at the file level. STGEN also outperforms all three baselines in terms of RSD on both Llama3.1 and GPT-4o. This demonstrates that stressful test cases generated by STGEN can better distinguish different code solutions than original correctness test cases and stressful test cases generated by baselines. Secondly, we find that the generator-based prompting method achieves higher RSD than other baselines. This verifies the effectiveness of generator test cases compared with raw test cases in time efficiency evaluation. However, the generator-based prompting method cannot well handle multiple parameters in function-level programs by achieving an accuracy of only 86.91%. STGEN mitigates this problem by generating expression test cases that follow the formats of raw test cases but introduce small expressions to represent each input. As a result, the expression test cases generated by STGEN for function-level code solutions outperform the generator-based prompting method by 19.25% and 32.27% in terms of RSD on Llama3.1 and GPT-4o, respectively.

Answer to RQ2: With knowledge enrichment and early validation by contracts, STGEN is quite effective in generating correct stressful test cases with an accuracy of about 99% and line coverage of about 96%. The expression and generator test cases generated by STGEN can better distinguish different code solutions' time efficiency with an RSD of up to 28.20% on GPT-4o.

5.3 RQ3: Time Efficiency of Code Generated by LLMs

Based on COFFE, we evaluate the time efficiency of code generated by different LLMs. We select ten popular open-source LLMs and four popular closed-source LLMs, as shown in Table 3. We show the Pass@1, efficient@1, and speedup of all LLMs on COFFE in Table 6.

Overall Time Efficiency. To evaluate the time efficiency of code generated by different models, we study the efficient@1 and speedup. We use efficient@1 to evaluate the probability of an LLM to generate a correct code solution faster than the best ground truth solution and speedup to evaluate how fast the correctly generated code solutions are compared with the best ground truth solutions. From Table 6, we identify that DeepSeek V2 Coder obtains the highest efficient@1 of 46.97% at the function level and Llama3.1 obtains the highest efficient@1 of 46.51% at the file level. As for the speedup, GPT-4o achieves the highest speedup of 8.28 at the function level, and Mixtral obtains the highest speedup of 1.43 at the file level.

Table 6. The correctness and time efficiency of code solutions generated by LLMs in Table 3 on COFFE. Efficient@1 and pass@1 are calculated upon all instances in COFFE, and speedup is calculated on correct solutions generated by models. Models highlighted in gray are closed-source models. “ Δ ” indicates the difference of efficient@1 and pass@1 in percentage ($100\% - \text{efficient@1} / \text{pass@1}$).

Model	Size	Function-level			File-level		
		Efficient@1 (Δ)	Speedup	Pass@1	Efficient@1 (Δ)	Speedup	Pass@1
Phi3	3.8B	26.65 (39%)	2.59	43.47	7.36 (67%)	0.08	22.63
MagicCoder	DS-6.7B	21.90 (32%)	3.04	32.41	12.02 (48%)	0.10	22.91
	CL-7B	29.82 (36%)	3.41	46.48	5.04 (68%)	0.14	15.92
CodeLlama	7B	26.65 (31%)	2.49	38.69	4.26 (51%)	0.95	8.66
	13B	25.60 (39%)	1.03	41.71	1.16 (48%)	1.02	2.23
	34B	40.37 (38%)	3.51	64.74	22.87 (57%)	0.09	53.63
Llama3	8B	27.70 (35%)	3.91	42.46	0.00 (100%)	0.21	0.84
	70B	40.90 (39%)	3.30	67.59	38.76 (44%)	0.14	68.99
StarCoder	15B	38.52 (37%)	3.52	61.31	21.71 (58%)	0.10	51.11
WizardCoder	15B	28.76 (41%)	1.95	48.49	10.08 (51%)	0.07	20.67
Mixtral	8 $\times$ 7B	25.59 (43%)	5.14	44.72	8.53 (63%)	1.43	22.91
DeepSeek V2	236B	46.70 (40%)	2.79	78.39	41.09 (54%)	0.18	89.94
DeepSeek V2 Coder	236B	46.97 (41%)	2.53	79.90	42.25 (46%)	0.44	78.77
Llama3.1	405B	39.58 (41%)	3.21	67.34	46.51 (48%)	0.90	89.11
Claude 3.5 Sonnet	-	43.54 (44%)	4.90	77.64	39.15 (55%)	0.23	86.59
Gemini 1.5 Pro	-	45.12 (40%)	1.76	75.38	42.64 (43%)	0.16	75.44
ChatGPT	-	37.73 (45%)	2.46	68.19	39.15 (48%)	0.12	75.98
GPT-4o	-	44.59 (43%)	8.28	77.64	43.02 (53%)	1.11	90.78

Finding 1: DeepSeek V2 Coder and Llama3.1 have the highest probability of generating efficient code solutions with an efficient@1 of 46.97% and 46.51%, respectively. GPT-4o and Mixtral generate the most efficient code solutions with a speedup of 8.28 and 1.43, respectively.

Correctness vs. Time Efficiency When comparing the correctness and time efficiency of code solutions generated by current LLMs, we find that the best efficient@1 are 46.97% and 46.51%, at function level and file level, respectively, which are much lower than the best pass@1 of 79.90% and 90.78%. This indicates that almost half of the correctly generated code solutions are sub-optimal since they are less efficient than ground truth solutions. Furthermore, the speedups achieved by most LLMs in file-level code generation are lower than 1.0, and some LLMs even obtain a speedup of lower than 0.1, indicating their generated code solutions are 10 $\times$ slower than ground truth solutions. This suggests that efficient code generation is a great challenge for current LLMs despite their remarkable performance on correct code generation.

Finding 2: The performance of current LLMs drops significantly in efficient code generation with the best efficient@1 of 46.97% and 46.51% at the function level and file level, compared with

that in correct code generation with the best Pass@1 of 79.9% and 90.78%. This indicates that the code solutions generated by current LLMs are correct but not time-efficient.

Function-level Code Generation vs. File-level Code Generation. In function-level code generation, we observe that all LLMs achieve a speedup larger than 1.0, indicating that the code solutions generated by current LLMs are more efficient than ground truths. However, only three LLMs achieve a speedup larger than 1.0 in the file-level code generation. This suggests that current LLMs cannot generate faster code solutions than existing solutions in COFFE. Besides, the efficient@1 achieved by current LLMs at the function level is also better than that achieved by current LLMs at the file level. For example, the efficient@1 of Phi3 drops by 72.38% from function-level to file-level code generation. Furthermore, the performance drop from pass@1 to efficient@1 in function-level code generation is 30%~45%, smaller than 40%~100% in file-level code generation. This indicates that current LLMs perform much worse in file-level efficient code generation than function-level efficient code generation.

Finding 3: Compared with function-level code generation, code solutions generated by current LLMs are less efficient in file-level code generation, evidenced by the significantly lower speedup, lower efficient@, and larger performance drop from pass@1 to efficient@1.

Impacts of Different Model Sizes. To study the impacts of different model sizes on the time efficiency of code solutions generated by current LLMs, we observe the changes of pass@1 and efficient@1 from smaller LLMs to larger LLMs. In both function-level and file-level code generation, we find that larger LLMs can generally generate more correct solutions, evidenced by higher Pass@1 obtained by larger LLMs. However, larger LLMs do not always generate more efficient code solutions. For example, in function-level code generation, CodeLlama-34b achieves an efficient@1 of 40.37%, which is quite close to the efficient@1 of 40.90% achieved by Llama3-70b and 39.58% achieved by Llama3.1-405b, but Llama3.1-405b is more than 10× larger than CodeLlama-34b. In file-level code generation, Llama3-70b achieves an efficient@1 of 38.76%, which is also quite close to the efficient@1 of 41.09% achieved by DeepSeek V2, and 42.25% achieved by Deep Seek V2 Coder, but DeepSeek V2 is more than 3× larger than Llama3-70b.

Finding 4: Larger LLMs generally perform better in terms of Pass@1 but do not significantly outperform smaller LLMs in terms of efficient@1, indicating larger parameter sizes of current LLMs do not contribute much to efficient code generation.

In summary, based on the experiment results of 14 popular LLMs on COFFE, we study the time efficiency of function-level and file-level code solutions generated by the LLMs in four aspects. We find efficient code generation much more challenging for current LLMs than correct code generation, especially for file-level efficient code generation. We also identify that larger LLMs do not always perform better on efficient code generation.

6 Implications

Based on the findings we conclude in Sec. 5, we provide some implications for researchers who build LLMs and practitioners who use LLMs in software development.

LLM Researchers. We identify that there is a large gap between correct code generation and efficient code generation. This indicates that the current LLM-generated code is correct but sub-optimal, and generating efficient code remains a great challenge, especially for file-level code generation. This challenge cannot be effectively mitigated by just increasing the model size of current LLMs. We recommend that LLM researchers consider the code structure and semantics

when improving the time efficiency of LLM-generated code. Besides, LLM researchers should also focus more on file-level code generation since current LLMs perform much worse on it than function-level code generation.

Software Practitioners. As LLMs are gradually adopted in software development in product environments, software practitioners face the problem of choosing LLMs. In function-level and file-level code generation, generally, code solutions generated by DeepSeek V2 Coder and Llama3.1-405b obtain the best time efficiency, respectively. However, we also find that some LLMs with middle sizes, such as Llama3-70b and CodeLlama-34b, achieve competitive performance. We recommend software practitioners adopt middle-sized LLMs to obtain similar performance on efficient code generation with much lower computational costs.

7 Threats to Validity

Our research may face the following threats to the internal and external validity.

7.1 Threats to Internal Validity

Performance Measurement. The time efficiency measurement of code solutions generated by LLMs can introduce errors. We propose to use CPU instruction count instead of execution time to improve the stability of measurements. However, there still exist factors such as specific code optimization techniques that introduce measurement errors. To mitigate the threats posed by the errors in time efficiency measurements, we conduct all measurements in dockers [19] to ensure that only one single process is running at the same time. Furthermore, we run the measurements for each code solution 12 times and remove the highest and lowest measurements before calculating the average metric. This could further reduce the errors introduced in a single measurement.

Baseline Implementation. Currently, there are no LLM-based stressful test case generation methods that could be compared with STGEN, so we modify three correctness test case generation methods as our baselines. However, such modifications may result in performance changes. To improve the validity of baselines, we run them on the most powerful and robust LLM GPT-4o [63]. Besides, we ask the baselines to generate 20 stressful test cases once and only choose the best 5 test cases for most evaluations except for accuracy. Therefore, we believe our implementations can represent the best performance of baselines.

7.2 Threats to External Validity

Adaptation to Different Programming Languages. While code generation is a general task for all programming languages, we mainly focus on the evaluation of Python code generation in this paper. The code generation performance of LLMs on other programming languages such as C++ and Java may be different from the experiment results we show in Sec. 5, as it could be affected by the syntax and coding styles. This threatens the validity of our experiment results in other programming languages. However, Python is the top 2 most popular programming language at GitHub [23] and is the major programming language used to build the code generation benchmarks [5, 8, 27, 28, 33, 43, 51, 91]. Besides, our stressful test case generation method STGEN is language-agnostic and fully based on LLMs to generate stressful test cases, we believe it could be easily extended to build benchmarks for other programming languages.

8 Related Work

8.1 LLMs for Code Generation

As a critical task to automate the software development process, code generation has drawn a lot of attention in both the academia and industry. At the beginning, encoder-decoder models such

as AlphaCode [43], CodeT5 [82], CodeRL [40], CodeT5+ [81] are directly trained on large code corpus and obtain good performance on code generation. Recently, decoder-only models such as Codex [9], CodeGen [59, 60], InCoder [22], CodeGeeX [93], SantaCoder [2], StarCoder [42, 52], WizardCoder [53], CodeLlama [71], MagicCoder [84], DeepSeek-Coder [26] show superior performance than encoder-decoder models on code generation. Besides, some general LLMs trained on multiple types of data, such as Llama3 [54], Llama3.1 [55], GPT-3.5 [61], GPT-4 [62] also demonstrate competitive or even better performance compared with code LLMs.

8.2 Code Generation Benchmarks

Correctness Benchmarks. There are many benchmarks designed for the correctness evaluation of code generated by LLMs. They provide contexts that indicate the functionality of the generated code and several test cases to evaluate the correctness of the generated code. The benchmarks are initially built from scratch by skilled developers and researchers. HumanEval [8] is a benchmark that contains 164 Python programming problems with function signatures and docstrings. MBPP [5] is a benchmark consisting of 974 basic Python programming problems with short functionality descriptions. It also provides a sanitized version with verified ground truth solutions that have 427 problems. In order to comprehensively evaluate the performance of LLMs, some benchmarks are built from code competition problems. APPS [28] contains 10,000 Python problems with different difficulty levels and diversified ground truth solutions for each problem. Code Contests [43] is a multi-lingual benchmark built from various competition sources and includes both correct and incorrect human solutions for each problem. Apart from Code Contests, there are also other multi-lingual benchmarks such as xCodeEval [38] and HumanEval-X [93]. The above-mentioned benchmarks focus on the evaluation of function-level or file-level code generation. There are some research efforts, such as RepoEval [91], RepoBench [51], SWE-Bench [36], and CrossCodeEval [18], devoted to the evaluation of the repo-level code generation performance.

Time Efficiency Benchmarks. Despite the well-explored evaluation for the correctness of code generated by LLMs, the time efficiency of code generated by LLMs is under-explored. Effibench [33] is the first benchmark designed for evaluating the time and memory efficiency of code generation. It selects efficiency-critical problems tagged “LeetCode” and prompts GPT-3.5 to generate test cases with different input sizes and data distribution. However, the problems in this benchmark are too difficult, so most open-source models cannot even generate correct solutions. Besides, it adopts execution time as the performance metric, which is unreliable to distinguish the efficiency of different code solutions. There is also some work [39, 78, 89] on traditional performance engineering, but they are not suitable for evaluating random responses from LLMs.

8.3 LLM-Based Test Case Generation

Apart from the advances in code generation, LLMs have also been demonstrated to improve software testing [17]. A lot of work has comprehensively evaluated the ability of LLMs on test case generation [37, 41, 57, 66, 72]. Most recently, Chen *et al.* [11] propose ChatUniTest, a unit test generation framework based on LLM by utilizing innovative mechanisms such as adaptive focal context and generation-validation-repair mechanisms. Liu *et al.* [49] propose a novel LLM-powered test oracle generation approach that combines LLMs and differential testing. Hossain *et al.* [31] propose TOGLL, a fine-tuned LLM on designed instruction prompts to generate test oracle for Java projects. Wang *et al.* [79] propose TestEval to generate test cases that cover certain lines, branches, and paths of the code under test. Despite the effectiveness of previous approaches on correctness test case generation, there is no work on stressful test case generation that aims to generate large test inputs to evaluate the time efficiency of the code under test. In this paper, we

propose a novel approach STGEN to generate stressful test cases for Python projects with high accuracy and coverage.

9 Conclusion

In this paper, we propose a new benchmark COFFE for the time efficiency evaluation of LLM-generated code. To address the challenges of existing correctness code generation benchmarks, we propose a novel stressful test case generation method STGEN that incorporates contracts and two test case formats to improve the accuracy. We also introduce a new time efficiency metric *efficient@k* based on CPU instruction count that stably evaluates both the correctness and time efficiency of code. Based on COFFE, we evaluate 14 popular LLMs and identify four important findings. We provide implications based on the findings for LLM researchers and software practitioners.

10 Data Availability

The code and data of STGEN and COFFE are available at <https://github.com/JohnnyPeng18/Coffe>.

Acknowledgment

The authors would like to thank the anonymous reviewers who have provided insightful and constructive comments on this paper. This work is supported by the National Nature Science Foundation of China (No. 62302437).

References

- [1] Marah I Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadallah, Hany Awadalla, Nguyen Bach, Amit Bahree, Arash Bakhtiari, et al. 2024. Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. *CoRR* abs/2404.14219 (2024). <https://doi.org/10.48550/ARXIV.2404.14219> arXiv:2404.14219
- [2] Loubna Ben Allal, Raymond Li, Denis Kocetkov, Chenghao Mou, Christopher Akiki, Carlos Muñoz Ferrandis, Niklas Muennighoff, Mayank Mishra, Alex Gu, Manan Dey, Logesh Kumar Umapathi, et al. 2023. SantaCoder: don't reach for the stars! *CoRR* abs/2301.03988 (2023). <https://doi.org/10.48550/ARXIV.2301.03988> arXiv:2301.03988
- [3] Anthropic. 2024. API reference provided by Anthropic. <https://docs.anthropic.com/en/api/getting-started>
- [4] Anthropic. 2024. Claude 3.5 Sonnet. <https://www.anthropic.com/news/claude-3-5-sonnet>
- [5] Jacob Austin, Augustus Odena, Maxwell I. Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. *CoRR* abs/2108.07732 (2021). arXiv:2108.07732 <https://arxiv.org/abs/2108.07732>
- [6] Ned Batchelder. 2024. The Coverage.py library. <https://github.com/nedbat/coveragepy>
- [7] Shreya Bhatia, Tarushi Gandhi, Dhruv Kumar, and Pankaj Jalote. 2023. Unit Test Generation using Generative AI : A Comparative Performance Analysis of Autogeneration Tools. *CoRR* abs/2312.10622 (2023). <https://doi.org/10.48550/ARXIV.2312.10622> arXiv:2312.10622
- [8] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harrison Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *CoRR* abs/2107.03374 (2021). arXiv:2107.03374 <https://arxiv.org/abs/2107.03374>
- [9] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harrison Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, et al. 2021. Evaluating Large Language Models Trained on Code. *CoRR* abs/2107.03374 (2021). arXiv:2107.03374 <https://arxiv.org/abs/2107.03374>
- [10] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching Large Language Models to Self-Debug. *CoRR* abs/2304.05128 (2023). <https://doi.org/10.48550/ARXIV.2304.05128> arXiv:2304.05128
- [11] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. ChatUniTest: A Framework for LLM-Based Test Generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering, FSE 2024, Porto de Galinhas, Brazil, July 15-19, 2024*. ACM, 572–576. <https://doi.org/10.1145/3663529.3663801>

- [12] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Eric Li, Xuezhi Wang, et al. 2022. Scaling Instruction-Finetuned Language Models. *CoRR* abs/2210.11416 (2022). <https://doi.org/10.48550/ARXIV.2210.11416> arXiv:2210.11416
- [13] The MITRE Corporation. 2024. Performance efficiency CWEs. <https://cwe.mitre.org/data/definitions/1132.html>
<https://cwe.mitre.org/data/definitions/1132.html>.
- [14] Deepmind. 2024. Gemini 1.5 Pro. <https://deepmind.google/technologies/gemini/pro/>
<https://deepmind.google/technologies/gemini/pro/>.
- [15] DeepSeek. 2024. DeepSeek API. <https://platform.deepseek.com/> <https://platform.deepseek.com/>.
- [16] DeepSeek-AI et al. 2024. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. *CoRR* abs/2405.04434 (2024). <https://doi.org/10.48550/ARXIV.2405.04434> arXiv:2405.04434
- [17] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2023. Large Language Models Are Zero-Shot Fuzzers: Fuzzing Deep-Learning Libraries via Large Language Models. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Seattle, WA, USA, July 17-21, 2023*. ACM, 423–435. <https://doi.org/10.1145/3597926.3598067>
- [18] Yangruibo Ding, Zijian Wang, Wasi Uddin Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. 2023. CrossCodeEval: A Diverse and Multilingual Benchmark for Cross-File Code Completion. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*. http://papers.nips.cc/paper_files/paper/2023/hash/920f2dcdd7d32ab2ba2f1970bc306af6-Abstract-Datasets_and_Benchmarks.html
- [19] Docker. 2024. Docker. <https://www.docker.com/> <https://www.docker.com/>.
- [20] Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K Lahiri. 2024. Can Large Language Models Transform Natural Language Intent into Formal Method Postconditions? *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1889–1912.
- [21] The Linux Foundation. 2024. The perf tool on linux. https://perf.wiki.kernel.org/index.php/Main_Page
https://perf.wiki.kernel.org/index.php/Main_Page.
- [22] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Scott Yih, Luke Zettlemoyer, and Mike Lewis. 2023. InCoder: A Generative Model for Code Infilling and Synthesis. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. <https://openreview.net/pdf?id=hQwb-lbM6EL>
- [23] Inc. GitHub. 2022. GitHub Octoverse report on programming languages. <https://octoverse.github.com/2022/top-programming-languages>
<https://octoverse.github.com/2022/top-programming-languages>.
- [24] Google. 2023. Sanitized version of MBPP benchmark released by Google. <https://huggingface.co/datasets/google-research-datasets/mbpp/viewer/sanitized/test>
<https://huggingface.co/datasets/google-research-datasets/mbpp/viewer/sanitized/test>.
- [25] Google. 2024. API reference provided by Google. <https://ai.google.dev/gemini-api/docs/models/gemini>
<https://ai.google.dev/gemini-api/docs/models/gemini>.
- [26] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming - The Rise of Code Intelligence. *CoRR* abs/2401.14196 (2024). <https://doi.org/10.48550/ARXIV.2401.14196> arXiv:2401.14196
- [27] Nam Le Hai, Dung Manh Nguyen, and Nghi D. Q. Bui. 2024. REPOEXEC: Evaluate Code Generation with a Repository-Level Executable Benchmark. *CoRR* abs/2406.11927 (2024). <https://doi.org/10.48550/ARXIV.2406.11927> arXiv:2406.11927
- [28] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring Coding Challenge Competence With APPS. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*. <https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/c24cd76e1ce41366a4bbe8a49b02a028-Abstract-round2.html>
- [29] Samuel Holt, Max Ruiz Luyten, and Mihaela van der Schaar. 2023. L2MAC: Large Language Model Automatic Computer for Unbounded Code Generation. *CoRR* abs/2310.02003 (2023). <https://doi.org/10.48550/ARXIV.2310.02003> arXiv:2310.02003
- [30] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, et al. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. <https://openreview.net/forum?id=VtmBAGCN7o>
- [31] Soneya Binta Hossain and Matthew B. Dwyer. 2024. TOGLL: Correct and Strong Test Oracle Generation with LLMs. *CoRR* abs/2405.03786 (2024). <https://doi.org/10.48550/ARXIV.2405.03786> arXiv:2405.03786

- [32] Dong Huang, Qingwen Bu, Jie M. Zhang, Michael Luck, and Heming Cui. 2023. AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation. *CoRR abs/2312.13010* (2023). <https://doi.org/10.48550/ARXIV.2312.13010> arXiv:2312.13010
- [33] Dong Huang, Jie M. Zhang, Yuhao Qing, and Heming Cui. 2024. EffiBench: Benchmarking the Efficiency of Automatically Generated Code. *CoRR abs/2402.02037* (2024). <https://doi.org/10.48550/ARXIV.2402.02037> arXiv:2402.02037
- [34] Deep Infra. 2024. Deep Infra API. <https://deepinfra.com/> <https://deepinfra.com/>.
- [35] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of Experts. *CoRR abs/2401.04088* (2024). <https://doi.org/10.48550/ARXIV.2401.04088> arXiv:2401.04088
- [36] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *CoRR abs/2310.06770* (2023). <https://doi.org/10.48550/ARXIV.2310.06770> arXiv:2310.06770
- [37] Rabimba Karanjai, Aftab Hussain, Md Rafiqul Islam Rabin, Lei Xu, Weidong Shi, and Mohammad Amin Alipour. 2024. Harnessing the Power of LLMs: Automating Unit Test Generation for High-Performance Computing. arXiv:2407.05202 [cs.SE] <https://arxiv.org/abs/2407.05202>
- [38] Mohammad Abdullah Matin Khan, M. Saiful Bari, Xuan Long Do, Weishi Wang, Md. Rizwan Parvez, and Shafiq R. Joty. 2023. xCodeEval: A Large Scale Multilingual Multitask Benchmark for Code Understanding, Generation, Translation and Retrieval. *CoRR abs/2303.03004* (2023). <https://doi.org/10.48550/ARXIV.2303.03004> arXiv:2303.03004
- [39] Christoph Laaber, Stefan Würsten, Harald C. Gall, and Philipp Leitner. 2020. Dynamically reconfiguring software microbenchmarks: reducing execution time without sacrificing result quality. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Virtual Event, USA) (ESEC/FSE 2020)*. Association for Computing Machinery, New York, NY, USA, 989–1001. <https://doi.org/10.1145/3368089.3409683>
- [40] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven Chu-Hong Hoi. 2022. CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*. http://papers.nips.cc/paper_files/paper/2022/hash/8636419dea1aa9fbd25fc4248e702da4-Abstract-Conference.html
- [41] Kefan Li and Yuan Yuan. 2024. Large Language Models as Test Case Generators: Performance Evaluation and Enhancement. *CoRR abs/2404.13340* (2024). <https://doi.org/10.48550/ARXIV.2404.13340> arXiv:2404.13340
- [42] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. StarCoder: may the source be with you! *CoRR abs/2305.06161* (2023). <https://doi.org/10.48550/ARXIV.2305.06161> arXiv:2305.06161
- [43] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-level code generation with AlphaCode. *Science* 378, 6624 (2022), 1092–1097. <https://doi.org/10.1126/science.abq1158> arXiv:<https://www.science.org/doi/pdf/10.1126/science.abq1158>
- [44] Jiawei Liu, Thanh Nguyen, Mingyue Shang, Hantian Ding, Xiaopeng Li, Yu Yu, Varun Kumar, and Zijian Wang. 2024. Learning Code Preference via Synthetic Evolution. arXiv:2410.03837 [cs.LG] <https://arxiv.org/abs/2410.03837>
- [45] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and LINGMING ZHANG. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Thirty-seventh Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=1qv610Cu7>
- [46] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. The MBPP Plus benchmark. <https://github.com/evalplus/evalplus/releases/tag/v0.2.1> <https://github.com/evalplus/evalplus/releases/tag/v0.2.1>
- [47] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and LINGMING ZHANG. 2024. EvalPlus Leaderboard. <https://evalplus.github.io/leaderboard.html> <https://evalplus.github.io/leaderboard.html>
- [48] Jiawei Liu, Songrun Xie, Junhao Wang, Yuxiang Wei, Yifeng Ding, and Lingming Zhang. 2024. Evaluating Language Models for Efficient Code Generation. arXiv:2408.06450 [cs.SE] <https://arxiv.org/abs/2408.06450>
- [49] Kaibo Liu, Yiyang Liu, Zhenpeng Chen, Jie M. Zhang, Yudong Han, Yun Ma, Ge Li, and Gang Huang. 2024. LLM-Powered Test Case Generation for Detecting Tricky Bugs. *CoRR abs/2404.10304* (2024). <https://doi.org/10.48550/ARXIV.2404.10304> arXiv:2404.10304
- [50] Shangqing Liu, Yu Chen, Xiaofei Xie, Jing Kai Siow, and Yang Liu. 2021. Retrieval-Augmented Generation for Code Summarization via Hybrid GNN. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net. <https://openreview.net/forum?id=zy-ty1gPxA>

- [51] Tianyang Liu, Canwen Xu, and Julian J. McAuley. 2023. RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems. *CoRR* abs/2306.03091 (2023). <https://doi.org/10.48550/ARXIV.2306.03091> arXiv:2306.03091
- [52] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. 2024. StarCoder 2 and The Stack v2: The Next Generation. *CoRR* abs/2402.19173 (2024). <https://doi.org/10.48550/ARXIV.2402.19173> arXiv:2402.19173
- [53] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2023. WizardCoder: Empowering Code Large Language Models with Evol-Instruct. *CoRR* abs/2306.08568 (2023). <https://doi.org/10.48550/ARXIV.2306.08568> arXiv:2306.08568
- [54] Meta. 2024. Llama3. <https://ai.meta.com/blog/meta-llama-3/> <https://ai.meta.com/blog/meta-llama-3/>.
- [55] Meta. 2024. Llama3.1. <https://ai.meta.com/blog/meta-llama-3-1/> <https://ai.meta.com/blog/meta-llama-3-1/>.
- [56] Niklas Muennighoff, Qian Liu, Armel Randy Zebaze, Qinkai Zheng, Binyuan Hui, Terry Yue Zhuo, Swayam Singh, Xiangru Tang, Leandro von Werra, and Shayne Longpre. 2024. OctoPack: Instruction Tuning Code Large Language Models. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. <https://openreview.net/forum?id=mw1PWNSWZP>
- [57] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin T. Vechev. 2024. Code Agents are State of the Art Software Testers. *CoRR* abs/2406.12952 (2024). <https://doi.org/10.48550/ARXIV.2406.12952> arXiv:2406.12952
- [58] Ansong Ni, Srini Iyer, Dragomir Radev, Veselin Stoyanov, Wen-Tau Yih, Sida I. Wang, and Xi Victoria Lin. 2023. LEVER: Learning to Verify Language-to-Code Generation with Execution. In *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 26106–26128. <https://proceedings.mlr.press/v202/ni23b.html>
- [59] Erik Nijkamp, Hiroaki Hayashi, Caiming Xiong, Silvio Savarese, and Yingbo Zhou. 2023. CodeGen2: Lessons for Training LLMs on Programming and Natural Languages. *CoRR* abs/2305.02309 (2023). <https://doi.org/10.48550/ARXIV.2305.02309> arXiv:2305.02309
- [60] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. https://openreview.net/pdf?id=iaYcJKpY2B_
- [61] OpenAI. 2022. ChatGPT. <https://openai.com/blog/chatgpt> <https://openai.com/blog/chatgpt>.
- [62] OpenAI. 2023. GPT-4 Technical Report. *CoRR* abs/2303.08774 (2023). <https://doi.org/10.48550/ARXIV.2303.08774> arXiv:2303.08774
- [63] OpenAI. 2024. GPT-4o. <https://openai.com/index/hello-gpt-4o/> <https://openai.com/index/hello-gpt-4o/>.
- [64] OpenAI. 2024. OpenAI API. <https://openai.com/api/> <https://openai.com/api/>.
- [65] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, et al. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*. http://papers.nips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html
- [66] Wendkūuni C. Ouédraogo, Kader Kaboré, Haoye Tian, Yewei Song, Anil Koyuncu, Jacques Klein, David Lo, and Tegawendé F. Bissyandé. 2024. Large-scale, Independent and Comprehensive study of the power of LLMs for test case generation. arXiv:2407.00225 [cs.SE] <https://arxiv.org/abs/2407.00225>
- [67] Md. Rizwan Parvez, Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Retrieval Augmented Code Generation and Summarization. In *Findings of the Association for Computational Linguistics: EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 16-20 November, 2021*. Association for Computational Linguistics, 2719–2734. <https://doi.org/10.18653/V1/2021.FINDINGS-EMNLP.232>
- [68] David A. Patterson and John L. Hennessy. 2012. *Computer Organization and Design - The Hardware / Software Interface (Revised 4th Edition)*. Academic Press. <http://www.elsevierdirect.com/product.jsp?isbn=9780123747501>
- [69] Yun Peng, Akhilesh Deepak Gotmare, Michael Lyu, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. 2024. PerfCodeGen: Improving Performance of LLM Generated Code with Execution Feedback. arXiv:2412.03578 [cs.SE] <https://arxiv.org/abs/2412.03578>
- [70] Tal Ridnik, Dedy Kredo, and Itamar Friedman. 2024. Code Generation with AlphaCodium: From Prompt Engineering to Flow Engineering. *CoRR* abs/2401.08500 (2024). <https://doi.org/10.48550/ARXIV.2401.08500> arXiv:2401.08500
- [71] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code Llama: Open Foundation Models for Code. *CoRR* abs/2308.12950 (2023). <https://doi.org/10.48550/ARXIV.2308.12950> arXiv:2308.12950
- [72] Malik Abdul Sami, Zeeshan Rasheed, Muhammad Waseem, Zheyang Zhang, Tomas Herda, and Pekka Abrahamsson. 2024. A Tool for Test Case Scenarios Generation Using Large Language Models. *CoRR* abs/2406.07021 (2024). <https://doi.org/10.48550/ARXIV.2406.07021> arXiv:2406.07021

- [73] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *IEEE Transactions on Software Engineering* 50, 1 (2024), 85–105. <https://doi.org/10.1109/TSE.2023.3334955>
- [74] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*. http://papers.nips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html
- [75] Alexander G Shypula, Aman Madaan, Yimeng Zeng, Uri Alon, Jacob R. Gardner, Yiming Yang, Milad Hashemi, Graham Neubig, Parthasarathy Ranganathan, Osbert Bastani, and Amir Yazdanbakhsh. 2024. Learning Performance-Improving Code Edits. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=ix7rLVHXyY>
- [76] Matt Stuchlik, Bruno P. Kinoshita, and Donald Lee. 2024. The Cirron library. <https://github.com/s7nfo/Cirron>
- [77] Hongjin Su, Shuyang Jiang, Yuhang Lai, Haoyuan Wu, Boao Shi, Che Liu, Qian Liu, and Tao Yu. 2024. ARKS: Active Retrieval in Knowledge Soup for Code Generation. *CoRR* abs/2402.12317 (2024). <https://doi.org/10.48550/ARXIV.2402.12317> arXiv:2402.12317
- [78] Luca Traini, Vittorio Cortellessa, Daniele Di Pompeo, and Michele Tucci. 2023. Towards effective assessment of steady state performance in Java software: are we there yet? *Empirical Softw. Engg.* 28, 1 (Jan. 2023), 57 pages. <https://doi.org/10.1007/s10664-022-10247-x>
- [79] Wenhan Wang, Chenyuan Yang, Zhijie Wang, Yuheng Huang, Zhaoyang Chu, Da Song, Lingming Zhang, An Ran Chen, and Lei Ma. 2024. TESTEVAL: Benchmarking Large Language Models for Test Case Generation. *CoRR* abs/2406.04531 (2024). <https://doi.org/10.48550/ARXIV.2406.04531> arXiv:2406.04531
- [80] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024. Executable Code Actions Elicit Better LLM Agents. *CoRR* abs/2402.01030 (2024). <https://doi.org/10.48550/ARXIV.2402.01030> arXiv:2402.01030
- [81] Yue Wang, Hung Le, Akhilesh Gotmare, Nghi D. Q. Bui, Junnan Li, and Steven C. H. Hoi. 2023. CodeT5+: Open Code Large Language Models for Code Understanding and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*. Association for Computational Linguistics, 1069–1088. <https://doi.org/10.18653/V1/2023.EMNLP-MAIN.68>
- [82] Yue Wang, Weishi Wang, Shafiq R. Joty, and Steven C. H. Hoi. 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*. Association for Computational Linguistics, 8696–8708. <https://doi.org/10.18653/V1/2021.EMNLP-MAIN.685>
- [83] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. 2022. Emergent Abilities of Large Language Models. *Trans. Mach. Learn. Res.* 2022 (2022). <https://openreview.net/forum?id=yzkSU5zdWd>
- [84] Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. 2023. Magicoder: Source Code Is All You Need. *CoRR* abs/2312.02120 (2023). <https://doi.org/10.48550/ARXIV.2312.02120> arXiv:2312.02120
- [85] Papers with Code. 2024. The Leaderboard of APPS benchmark on Papers with Code. <https://paperswithcode.com/sota/code-generation-on-apps> <https://paperswithcode.com/sota/code-generation-on-apps>
- [86] Papers with Code. 2024. The Leaderboard of the Code Contests benchmark. <https://paperswithcode.com/sota/code-generation-on-codecontests>
- [87] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *CoRR* abs/2405.15793 (2024). <https://doi.org/10.48550/ARXIV.2405.15793> arXiv:2405.15793
- [88] Xiao Yu, Lei Liu, Xing Hu, Jacky Wai Keung, Jin Liu, and Xin Xia. 2024. Where Are Large Language Models for Code Generation on GitHub? arXiv:2406.19544 [cs.SE] <https://arxiv.org/abs/2406.19544>
- [89] Dmitrijs Zaporanuks, Milan Jovic, and Matthias Hauswirth. 2009. Accuracy of performance counter measurements. In *2009 IEEE International Symposium on Performance Analysis of Systems and Software*. 23–32. <https://doi.org/10.1109/ISPASS.2009.4919635>
- [90] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*. Association for Computational Linguistics, 2471–2484. <https://doi.org/10.18653/V1/2023.EMNLP-MAIN.151>
- [91] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*.

- Association for Computational Linguistics, 2471–2484. <https://doi.org/10.18653/V1/2023.EMNLP-MAIN.151>
- [92] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. *CoRR* abs/2404.05427 (2024). <https://doi.org/10.48550/ARXIV.2404.05427> arXiv:2404.05427
- [93] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Zihan Wang, Lei Shen, Andi Wang, Yang Li, Teng Su, Zhilin Yang, and Jie Tang. 2023. CodeGeeX: A Pre-Trained Model for Code Generation with Multilingual Evaluations on HumanEval-X. *CoRR* abs/2303.17568 (2023). <https://doi.org/10.48550/ARXIV.2303.17568> arXiv:2303.17568
- [94] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2023. Language Agent Tree Search Unifies Reasoning Acting and Planning in Language Models. *CoRR* abs/2310.04406 (2023). <https://doi.org/10.48550/ARXIV.2310.04406> arXiv:2310.04406
- [95] Shuyan Zhou, Uri Alon, Frank F. Xu, Zhengbao Jiang, and Graham Neubig. 2023. DocPrompting: Generating Code by Retrieving the Docs. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. <https://openreview.net/forum?id=ZTCxT2t2Ru>
- [96] Qihao Zhu, Daya Guo, Zhihong Shao, Dejian Yang, Peiyi Wang, Runxin Xu, Y Wu, Yukun Li, Huazuo Gao, Shirong Ma, et al. 2024. DeepSeek-Coder-V2: Breaking the Barrier of Closed-Source Models in Code Intelligence. *arXiv preprint arXiv:2406.11931* (2024).

Received 2024-09-13; accepted 2025-01-14